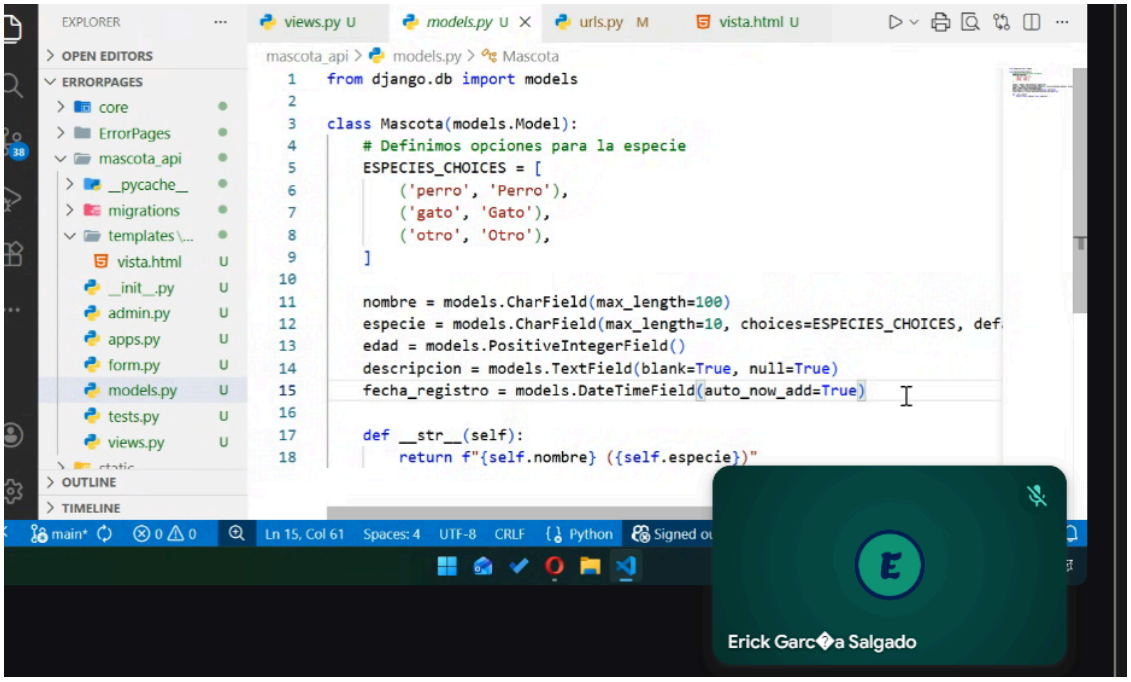
Desarrollo de una api con DJANGO

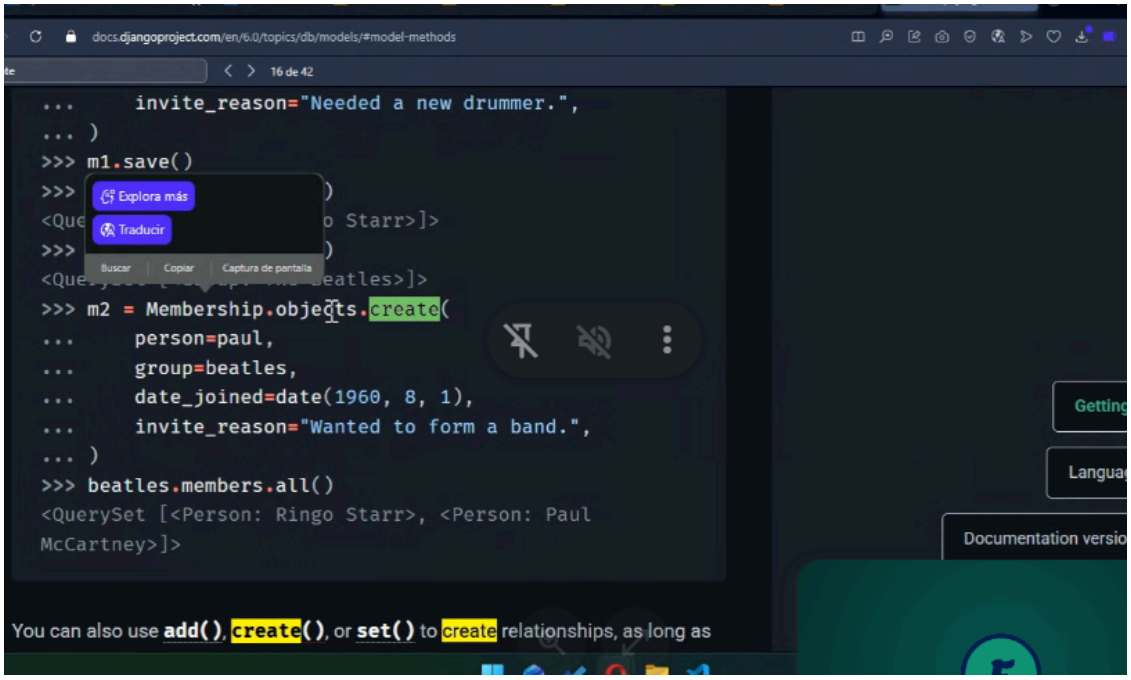


mascota api es una aplicación.

Tuplas, arreglos de arreglos. ESPECIES_CHOICES es una tupla.

Eso es el modelo. Un modelo es una clase que está ligada directamente con la BD.

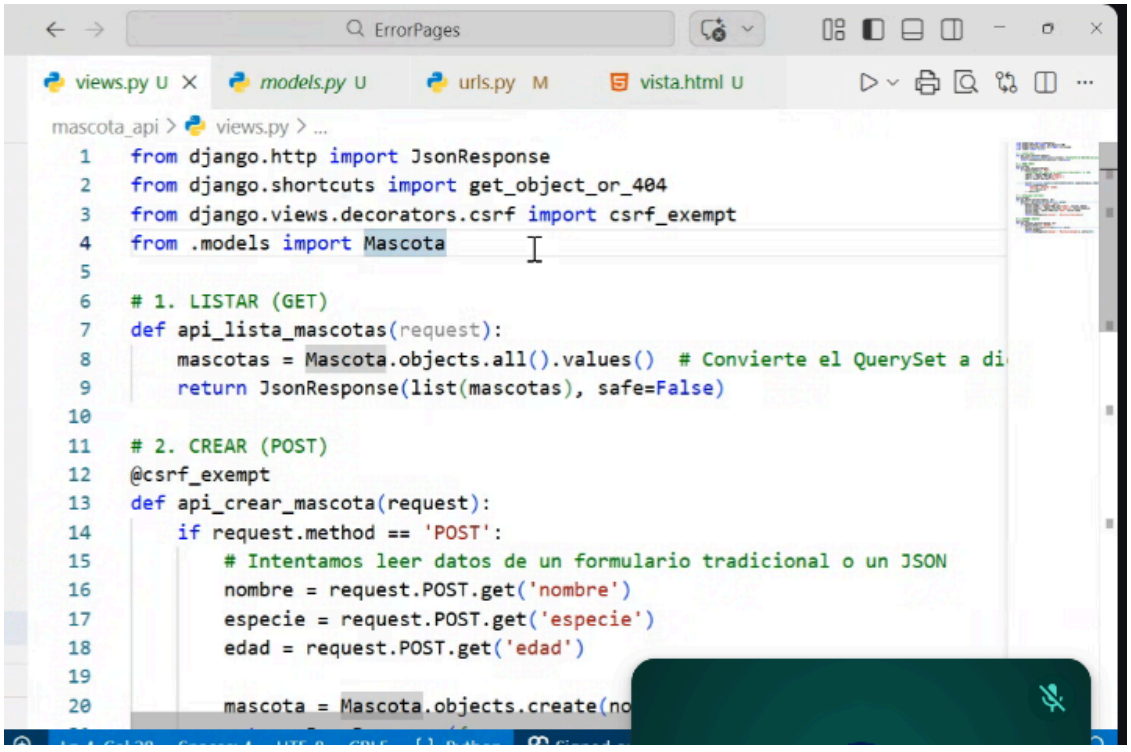
Models está en django.db, se puede consultar la documentación.



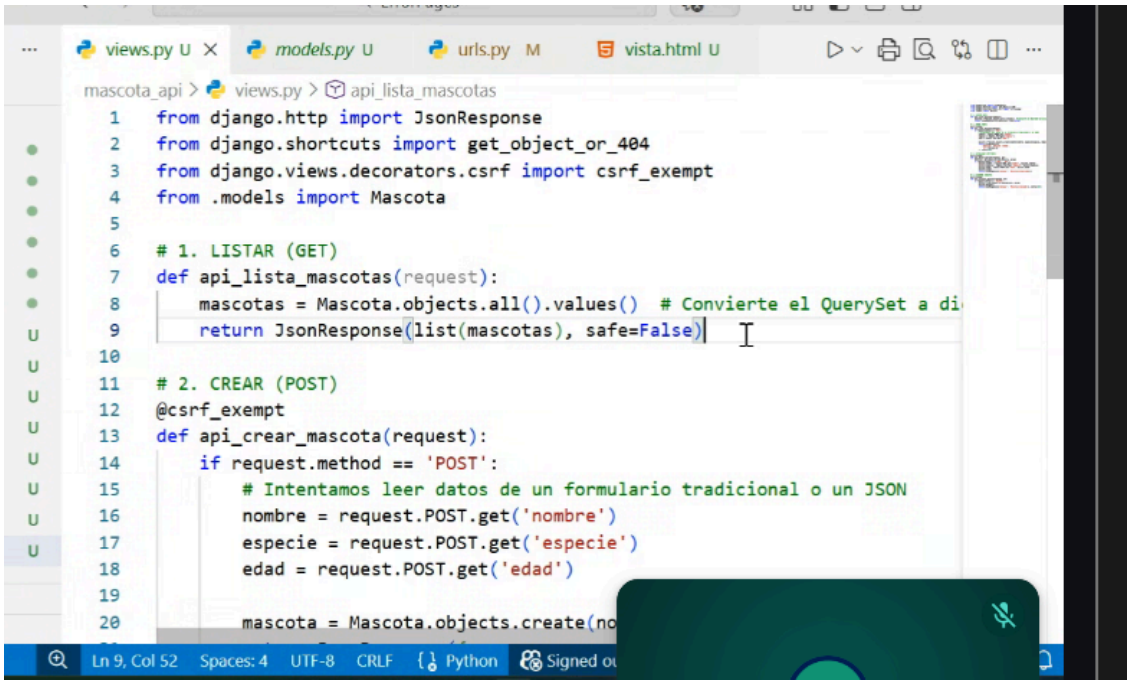
Así trataremos el CRUD de registro.

VIEWS.PY

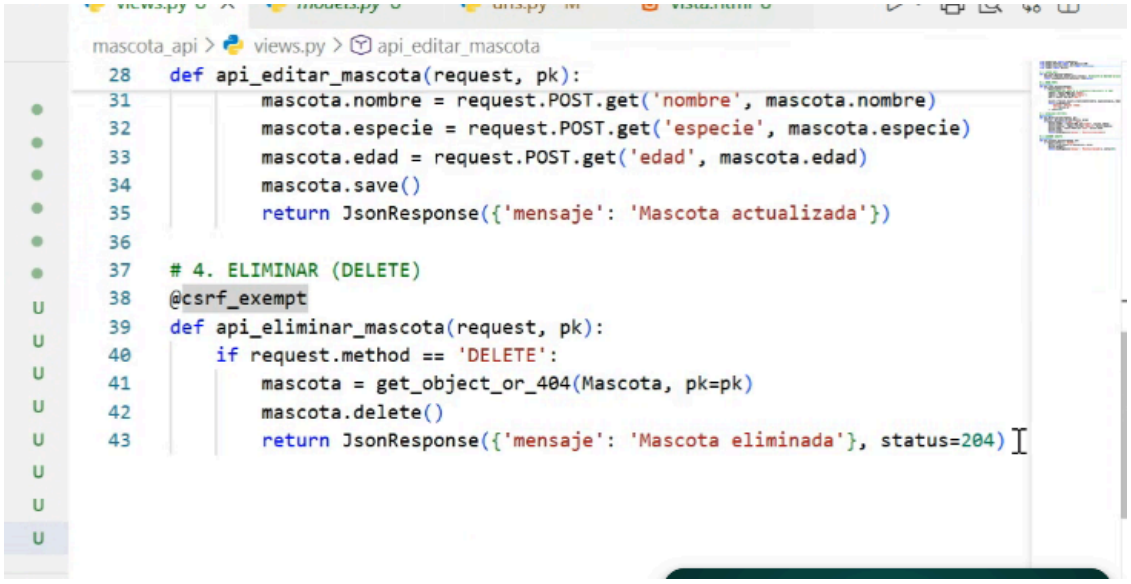
Lo usaremos para crear nuestra api “artesanal” porque se creará desde 0



APRENDER PARA EXAMEN



Necesito que me hagas los métodos CRUD de X entidad (examen)



CRUD

Expliacación

```python
from django.http import JsonResponse
from django.shortcuts import get_object_or_404
from django.views.decorators.csrf import csrf_exempt
from .models import Mascota

# 1. LISTAR (GET)
def api_lista_mascotas(request):
    mascotas = Mascota.objects.all().values()  # Convierte el QuerySet a di
    return JsonResponse(list(mascotas), safe=False)

# 2. CREAR (POST)
@csrf_exempt
def api_crear_mascota(request):
    if request.method == 'POST':
        # Intentamos leer datos de un formulario tradicional o un JSON
        nombre = request.POST.get('nombre')
        especie = request.POST.get('especie')
        edad = request.POST.get('edad')

        mascota = Mascota.objects.create(nombre=nombre, especie=especie, ed
```

JsonResponse es lo que usaremos a partir de ahora ya que no renderizaremos nada

get_object_or_404, es una funcion que ocuparemos

csrf_exempt es para que la función de POST no necesite una validación de csrf, es un decorador.

MÉTODO GET

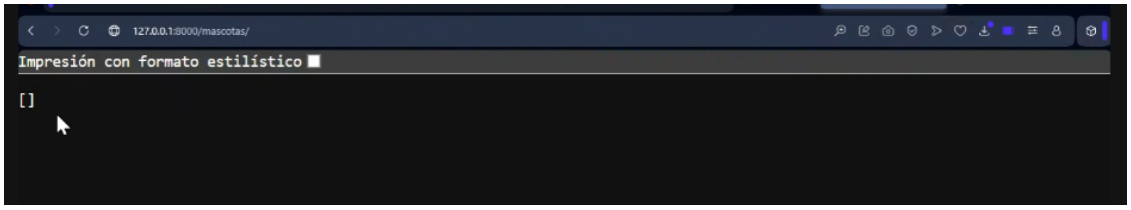
Se define una función y necesito un request, aunque solo lo ponemos como requerimiento de una función, no lo usaremos.

una variable, mascotas que dice: para el modelo Mascota guarda todos los valores de mascotas. Se convierte en un diccionario y se puede mandar un jsonrepsonse

En resumen: Se obtienen todos los valores de la bd y se devuelven como un jsonrepsonse que se formatea como una lista de python (list(mascotas).

```
# 1. LISTAR (GET)
def api_lista_mascotas(request):
    mascotas = Mascota.objects.all().values()  # Convierte el QuerySet a diccionarios
    return JsonResponse(list(mascotas), safe=False)
```

No quick fixes available



Ahorita no hay

Nota:

```
from core import views as core
from mascota_api import views as mascotas

urlpatterns = [
    path('',core.index,name='index'),
    path('nuevo/',core.nuevo, name='nuevo'),
    path('formulario/', core.contacto_view, name='formulario')

    path('mascotas/', mascotas.api_lista_mascotas, name='lista_mascotas'),
    path('mascotas/nueva/', mascotas.api_crear_mascota, name='crear_mascota'),
    path('mascotas/editar/<int:pk>/', mascotas.api_editar_mascota, name='editar_mascota'),
    path('mascotas/eliminar/<int:pk>/', mascotas.api_eliminar_mascota, name='eliminar_mascota'),

]
```

En el archivo urls.py tenemos definido esto (son como las rutas).

MÉTODO POST.

Aquí si hay request xq mandamos info del front al back

Se extraen los datos, e inicializamos una Mascota con los valores obtenidos.

con el .create.

PARA QUE LO REGISTRE EN EL MODELO

Al final se hace un return con el JsonResponse con el mensaje y el status etc.

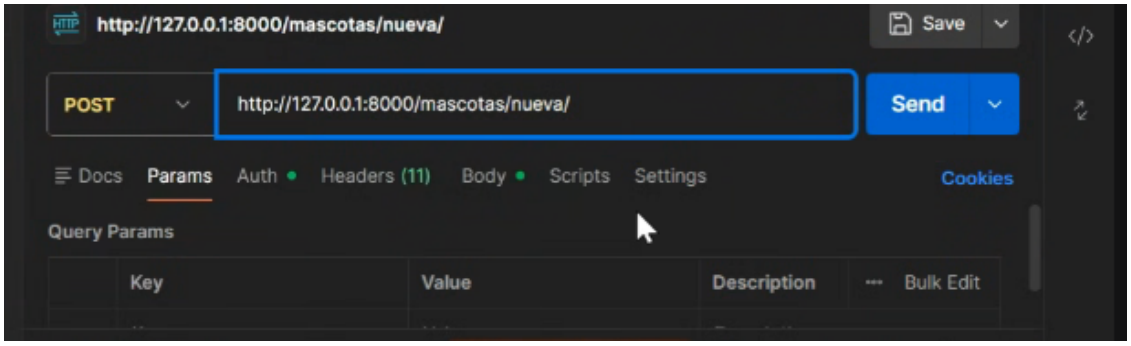
```python
# 2. CREAR (POST)
@csrf_exempt
def api_crear_mascota(request):
    if request.method == 'POST':
        # Intentamos leer datos de un formulario tradicional o un JSON
        nombre = request.POST.get('nombre')
        especie = request.POST.get('especie')
        edad = request.POST.get('edad')

        mascota = Mascota.objects.create(nombre=nombre, especie=especie, edad=edad)
        return JsonResponse({
            'mensaje': 'Mascota creada',
            'id': mascota.id
        }, status=201)
```

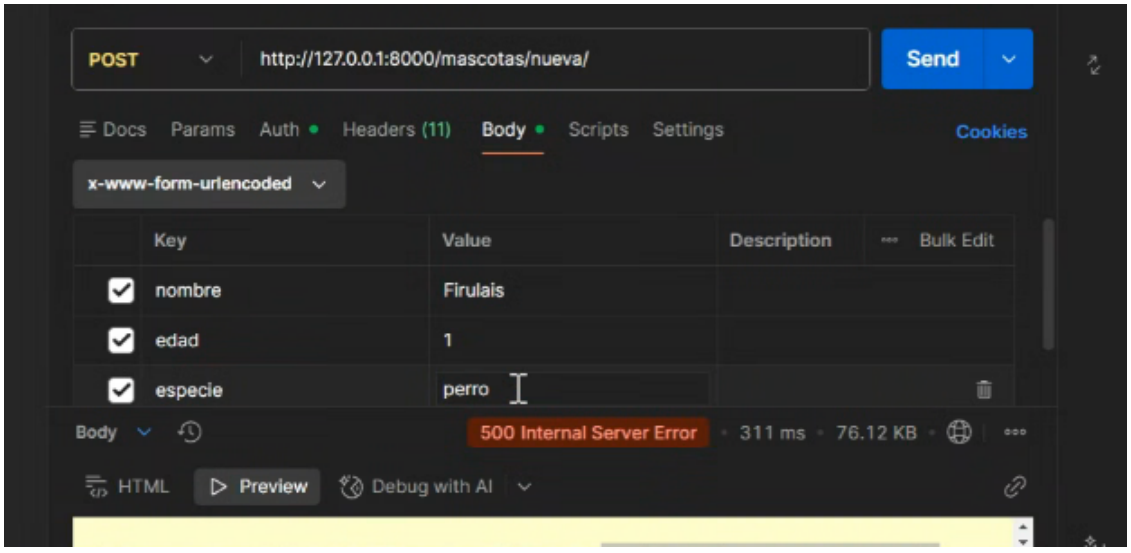
Hay otros 2 parámetros pero son opcionales.

Según la ruta nos metemos en postman y

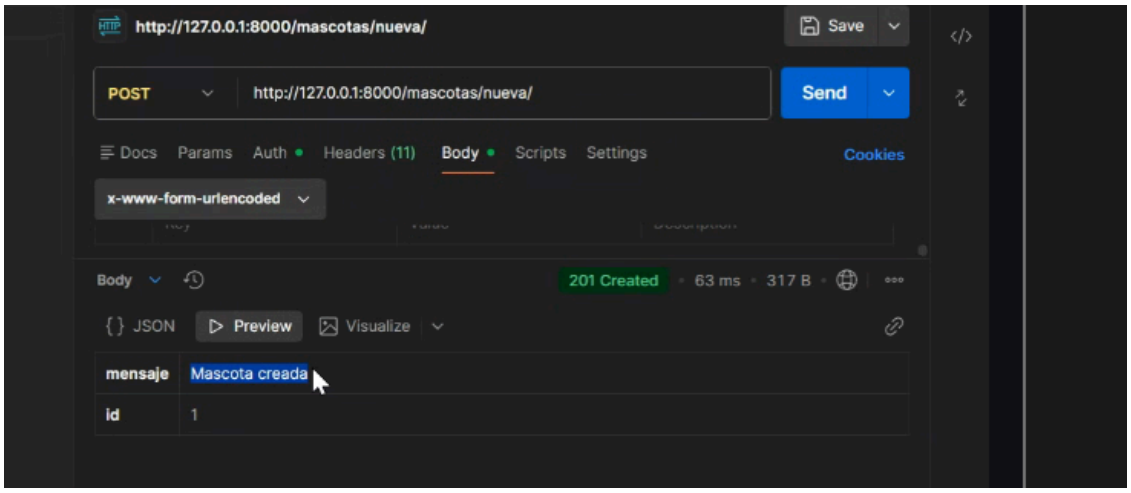
Es importante poner la última diagonal SIEMPRE



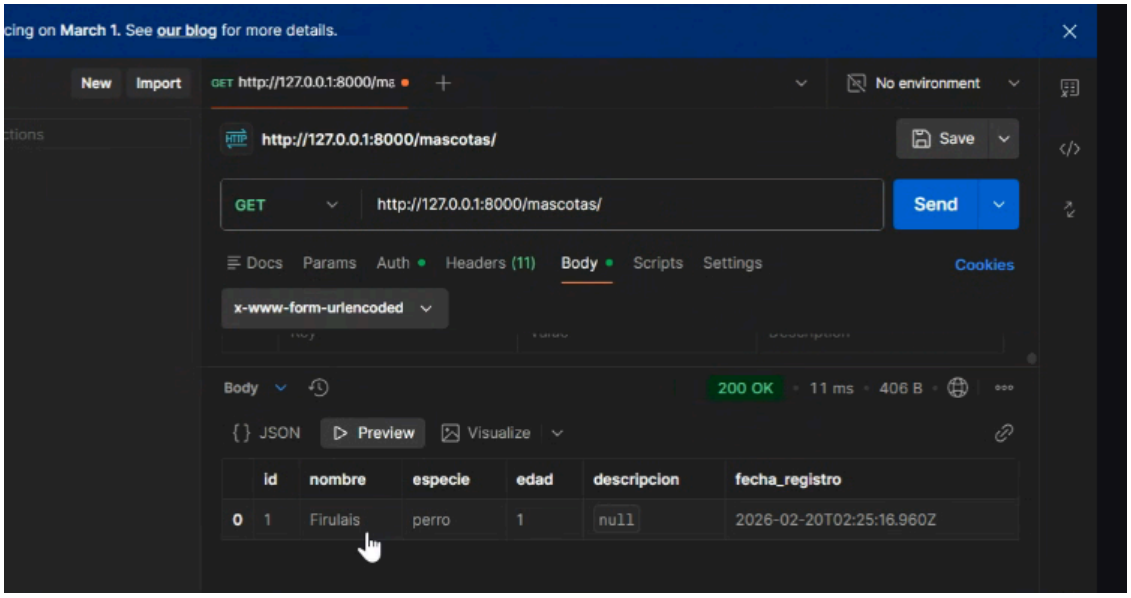
Llenamos el body para mandar la res

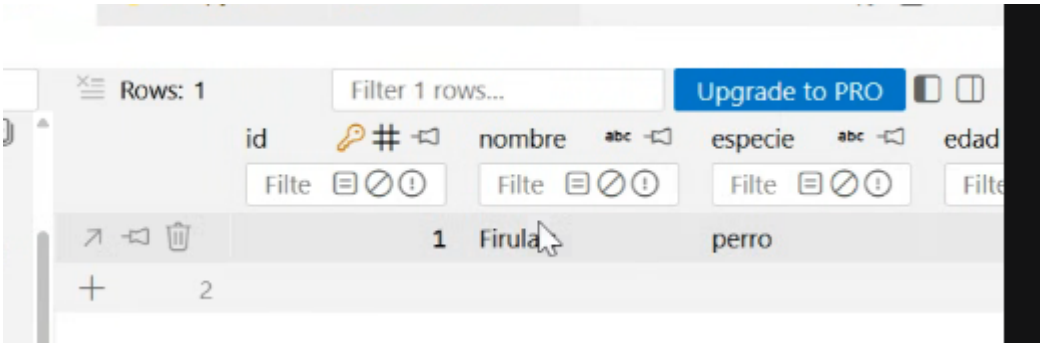


Y ya le damos send



Y podemos probar en get que ya está





El examen es para hacer algo así. y tokens de json.

Actualizar PUT

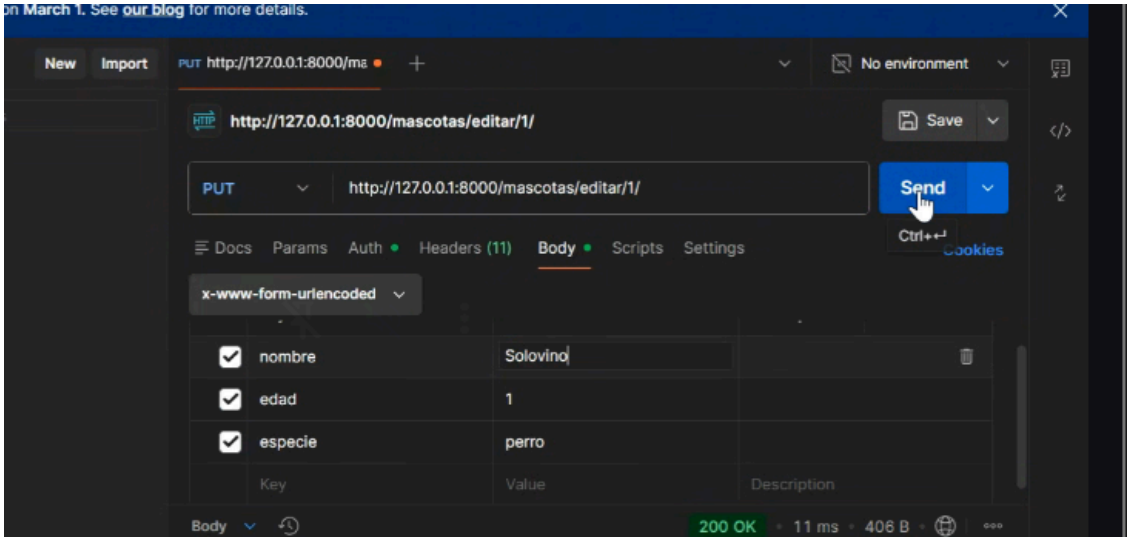
La primera linea es para encontrar a la mascota (ver si existe=

para eso usamos el get_objetct_or

```
24          }, status=201)
25
26  # 3. ACTUALIZAR (PUT/POST)
27  @csrf_exempt
28  def api_editar_mascota(request, pk):
29      mascota = get_object_or_404(Mascota, pk=pk)
30      if request.method == 'PUT':
31          mascota.nombre = request.PUT.get('nombre', mascota.nombre)
32          mascota.especie = request.PUT.get('especie', mascota.especie)
33          mascota.edad = request.PUT.get('edad', mascota.edad)
34          mascota.save()
35          return JsonResponse({'mensaje': 'Mascota actualizada'})
36
37  # 4. ELIMINAR (DELETE)
38  @csrf_exempt
39  def api_eliminar_mascota(request, pk):
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL python

Y ya probamos



Delete

```python
        return JsonResponse({'mensaje': 'Mascota actualizada'})

# 4. ELIMINAR (DELETE)
@csrf_exempt
def api_eliminar_mascota(request, pk):
    if request.method == 'DELETE':
        mascota = get_object_or_404(Mascota, pk=pk)
        mascota.delete()
        return JsonResponse({'mensaje': 'Mascota eliminada'}, status=204)
```